

MAE403 / MAE5005 Computational Fluid Dynamics

计算流体力学

Course syllabus

Benefits of the course

Two coding examples

Computer representation of real numbers: bits & bytes

Computer machine precision and roundoff errors

Introduction to computational fluid dynamics

Computation as a third pillar for scientific discovery

Lian-Ping Wang (王连平教授)

Department of Mechanics and Aerospace Engineering, SUSTech, wanglp@sustech.edu.cn

Teaching Assistants: Ms. LONG Xing (龙杏), CoE North Building 738, 12432328@mail.sustech.edu.cn

Mr. YOU Jingxiang (游敬翔), CoE North Building 738, 12431154@mail.sustech.edu

ZHAO, Xiaozheng (赵晓争, Computer system administrator)

Course arrangements (Spring 2026):

2.28 (week 1): Lecture 1

3.02 (week 2): Lecture 2

3.09 (week 3): Lecture 3

3.16 (week 4): Lecture 4

3.23 (week 5): Lecture 5

3.30 (week 6): Lecture 6

4.06 (week 7): **Holiday (清明节)**

4.13 (week 8): **Midterm exam**

4.20 (week 9): Lecture 7

4.27 (week 10): Lecture 8

5.04 (week 11): **Holiday (劳动节/青年节)**

5.11 (week 12): Lecture 9

5.18 (week 13): Lecture 10

5.25 (week 14): Lecture 11

6.01 (week 15): Lecture 12

6.08-18 (Final Exam Week): **Final Exam**

Make-up classes for weeks 7 & 11, possible time?

February	Winter Break	2	3	4	5	6	7	8
	Winter Break	9	10	11	12	13	14	15
	Winter Break	16	17	18	19	20	21	22
	Week 1 Spring Semester	23	24	25	26	27	28	1
March	Week 2 Spring Semester	2	3	4	5	6	7	8
	Week 3 Spring Semester	9	10	11	12	13	14	15
	Week 4 Spring Semester	16	17	18	19	20	21	22
	Week 5 Spring Semester	23	24	25	26	27	28	29
April	Week 6 Spring Semester	30	31	1	2	3	4	5
	Week 7 Spring Semester	6	7	8	9	10	11	12
	Week 8 Exam Week	13	14	15	16	17	18	19
	Week 9 Exam Week	20	21	22	23	24	25	26
May	Week 10 Spring Semester	27	28	29	30	1	2	3
	Week 11 Spring Semester	4	5	6	7	8	9	10
	Week 12 Spring Semester	11	12	13	14	15	16	17
	Week 13 Spring Semester	18	19	20	21	22	23	24
	Week 14 Spring Semester	25	26	27	28	29	30	31
June	Week 15 Spring Semester	1	2	3	4	5	6	7
	Week 16 Exam Week	8	9	10	11	12	13	14
	Week 17 Exam Week	15	16	17	18	19	20	21

Course arrangements: other logistics (课程保障)

A course WeChat group: 2026Spring-计算流体力学

A course “Blackboard” website

<https://bb.sustech.edu.cn/> Advanced Computational Fluid Mechanics Spring 2026

[Course syllabus](#)

[First set of lecture notes](#)

[Reference textbooks](#)

[Homework 1](#)

.....

Coding and homework rules

1. For all homework problems that require coding, the code should be written in Fortran 90.

Fortran is a main language for efficient CFD codes.

Late in the semester, you are required to understand a Fortran NS solver code

It is a good idea that you learn multiple programming languages

I will show you a lot of small Fortran codes

Your TAs can help you if you have any coding issue

2. Homework problems

Homework assignments must be done by yourself, in order to really learn things

- I encourage you to use DeepSeek / ChatGPT to help you learn Fortran 90
- You may also use them to help you with your homework problems, **but you are responsible for all the mistakes**. You need to fully understand things.
- I will also incorporate DeepSeek into my teaching, as appropriate.

Popular CFD programming languages

Language	Pros & Cons	Notes
Fortran	Most popular for large-scale CFD research codes, high efficiency	Many CFD libraries are developed initially in Fortran
C / C++	Popular for commercial CFD packages	Good for designing software structure
Matlab	Good for <i>simple</i> CFD codes	Convenient for postprocessing
Python	Good for <i>simple</i> CFD codes	Convenient for postprocessing

1. Others: Julia, Rust,
2. To write user add-ons, Ansys CFX uses FORTRAN, Ansys Fluent uses C, OpenFOAM uses C++

“As for the results, it is possible to infer that among the programming languages the one with the best performance is FORTRAN.”

Other reasons for learning Fortran 90

Historical Significance: One of the oldest high-level programming languages specifically designed for scientific computing.

Performance: Efficient handling of numerical computations and array operations.

Legacy Codes: Many existing CFD codes are written in Fortran; understanding it allows students to work with established software.

Community Support: Extensive libraries and resources available for scientific applications.

A course WeChat group:
2026Spring-计算流体力学

Will do during the break

Use your real name
in the WeChat group

Provide a face picture (Homework 1)

Benefits of the course (1)

What is computational fluid dynamics?

1. Physical assumptions & governing principles:

mass conservation, momentum conservation

1st law of thermodynamics, 2nd law of thermodynamic
equation of state

2. Mathematical models:

continuum mechanics, boundary /initial conditions

3. Numerical algorithms:

finite difference, finite element, spectral, lattice-Boltzmann

.....

4. Realization of computation:

coding: Fortran, C, matlab,, code optimization

implementation: computers, OS, parallelization

5. Verification and Validation

6. Use and interpretation of the computer solution

Learning objectives

As a result of taking this course, I hope that you can better:

- ❖ understand key concepts and ideas (e.g., numerical viscosity, numerical dispersion error, numerical stability, computational efficiency) used in computational fluid dynamics;
- ❖ utilize the method of separation of variables to solve linear partial differential equations;
- ❖ determine the time and space accuracy of a finite difference or finite volume method;
- ❖ apply the Taylor expansion to diverse analyses in computational fluid dynamics;
- ❖ derive the modified equation for a particular numerical scheme;
- ❖ perform a stability analysis of a numerical scheme;
- ❖ understand numerical issues related to the advection equation and proper methods to evaluate flux at cell interface;
- ❖ write your own Fortran codes and run the codes on a Unix system
- ❖ Analyze results from computational models critically

What did we learned this semester? (Answers from students at the end of semester)

Theory

- ❖ Analytical solution of PDE
- ❖ Van-Neumann stability analysis
 - Complex variable analysis
- ❖ Taylor expansion / order of accuracy analysis
- ❖ Finite-difference representations
- ❖ Derivation of modified equation
- ❖ Numerical viscosity / phase dispersion errors
- ❖ Round-off errors / boundary condition errors
- ❖ Exact solution of finite-difference schemes
- ❖ Designing my own schemes
- ❖

Numerical schemes & solvers

- ❖ Adams-Bashforth scheme
- ❖ Crank-Nicolson scheme
- ❖ Lax-Wendroff scheme
- ❖ WENO reconstruction
- ❖ Fractional step method
- ❖ Factorization
- ❖ Limiters
- ❖

Tools

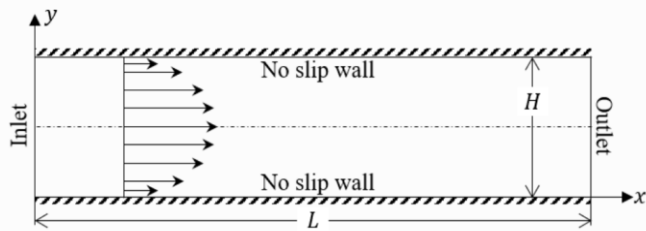
- ❖ Binary representation of real values
- ❖ Fortran programming skills
- ❖ Use contour plots to analyze stability
- ❖ Discrete Fourier transform
- ❖

Why Advanced Computational Fluid Dynamics?

- ❖ The course is a rigorous introduction of Computational Fluid Dynamics (严谨的科学)
There will be rigorous mathematics (Taylor expansion, method of separation of variables, Bessel functions, etc.) in the course
We learn CFD as a science, so we may advance CFD later on
- ❖ Training on combining a variety of skills (综合能力)
Physics, algorithms, coding by Fortran, Actually run codes on a Unix platform, write professional reports etc.
Proper mentality: Learn what we need, willingly
- ❖ A deep look at some relatively simple problems
I do not ask for too much prior knowledge, but I ask you to work hard
Come to each class (this is required)
- ❖ It is NOT because we teach the most cutting-edge CFD approaches

A few case-study problems

I. The unsteady planar Poiseuille flow



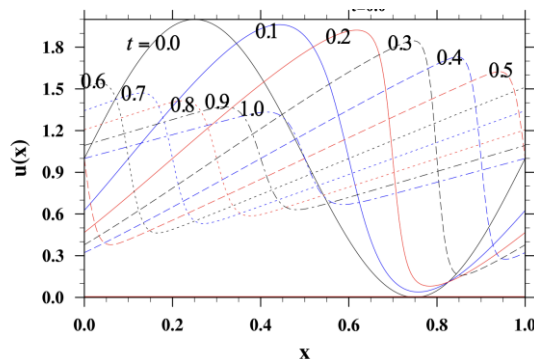
$$\frac{\partial u}{\partial t} = g + \nu \frac{\partial^2 u}{\partial y^2}$$

Model for flow through **pipeline**

II. The linear advection (wave equation)

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0$$

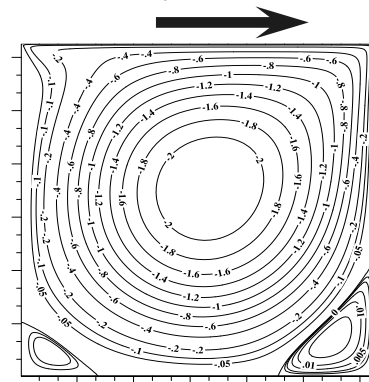
III. The Nonlinear Burgers Equation



$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

A simple 1D model of **nonlinear** fluid dynamics

IV. Two dimensional lid-driven cavity flow



$$\begin{aligned} \nabla \cdot \mathbf{u} &= 0 \\ \frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} &= -\frac{1}{\rho} \nabla p + \frac{\mu}{\rho} \nabla^2 \mathbf{u} \end{aligned}$$

The most popular CFD **benchmark** case

An example: development of a transient laminar flow in a channel

1. Physical assumptions & governing principles:

- ✧ *Unidirectional laminar* flow of *incompressible* fluid in a channel
- ✧ Initial velocity = 0
- ✧ The flow is driven by a body force (or constant pressure gradient) starting at $t=0$
- ✧ Governing equations:

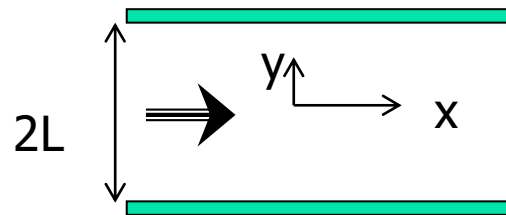
mass conservation – continuity equation

$$\nabla \cdot \vec{u} = 0$$

momentum equation – Navier-Stokes equations

$$\frac{\partial \vec{u}}{\partial t} + \vec{u} \cdot \nabla \vec{u} = -\frac{1}{\rho} \nabla p + \vec{g} + \frac{\mu}{\rho} \nabla^2 \vec{u}$$

- ✧ Boundary conditions: zero velocity on the walls



Physical parameters

channel half width: L
Body force per unit mass: g
Fluid density: ρ
fluid viscosity: μ

An example: development of a transient laminar flow in a channel

2. Mathematical model

$$\vec{u} = (u, v, w)$$

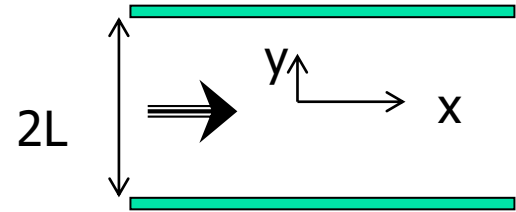
where $u = u(y, t)$, $v = 0$, $w = 0$

Simplified momentum equation

$$\frac{\partial u}{\partial t} = g_x + \frac{\mu}{\rho} \cdot \frac{\partial^2 u}{\partial y^2}$$

$$I.C. \quad u(y, t = 0) = 0$$

$$B.C.s \quad u(y = \pm L, t) = 0$$



Long-time solution implies

$$\rho g_x = \frac{8\mu u_0}{(2L)^2} = \frac{2\mu u_0}{L^2}$$

$u_0 = \text{maximum velocity at } y = 0$

This problem can be solved analytically by the method of separation of variables
& numerically by CFD

Analytical solution (the method of separation of variables)

$$\frac{u}{u_0} = \left(1 - \frac{y^2}{L^2}\right) - \sum_{k=0}^{\infty} \frac{4 \cdot (-1)^k}{[(k + 0.5)\pi]^3} \exp\left[-\frac{(k + 0.5)^2 \pi^2 \nu t}{L^2}\right] \cos\left[(k + 0.5)\pi \frac{y}{L}\right]$$

Note the format of the solution

$$\frac{u}{u_0} = f\left(\frac{y}{L}, \frac{t}{L^2/\nu}\right)$$

Clearly, the physical time scale of the problem is

$$L^2/\nu$$

Question: Why is this physical time scale important?

$$\frac{\partial u}{\partial t} = (g) + v \frac{\partial^2 u}{\partial y^2}$$

$$u(y, t=0) = 0$$

$$u(y = \pm L, t) = 0$$

$$\theta = \left(1 - \frac{y^2}{L^2}\right) - \frac{u}{u_0}$$

Transformation

$$u = u_0 \theta + u_0 \left(1 - \frac{y^2}{L^2}\right)$$

$$\Rightarrow \frac{\partial \theta}{\partial t} = v \frac{\partial^2 \theta}{\partial y^2} \text{ homogeneous}$$

$$\theta(y = \pm L, t) = 0$$

$$\theta(y, t=0) = \left(1 - \frac{y^2}{L^2}\right)$$

$$\theta(y, t) = Y(y) \cdot T(t)$$

$$Y T' = v Y'' T$$

$$\frac{T'(t)}{v T(t)} = \frac{Y''(y)}{Y(y)} = \text{Constant} = -\lambda^2$$

$$T(t) = C e^{-v \lambda^2 t}$$

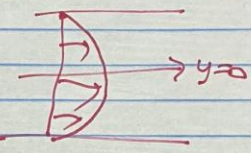
$$Y'' + \lambda^2 Y = 0$$

$$Y(y) = A_1 \cos(\lambda y) + A_2 \sin(\lambda y)$$

$y = \pm L$

$$A \sim C \cos(\lambda y) \cdot e^{-v \lambda^2 t}$$

$$\cos \lambda L = 0 \quad \lambda L = \frac{\pi}{2} + n\pi$$



$$\theta(y, t) = \sum C_n \cos\left(\frac{(2n+1)\pi y}{2L}\right) e^{-v \lambda_n^2 t}$$

$$I.C. \quad 1 - \frac{y^2}{L^2} = \sum_{n=0, 2, 4, \dots} C_n \cos\left(\frac{(2n+1)\pi y}{2L}\right)$$

$$\lambda_n = \frac{(2n+1)\pi}{2L}$$

$$\int_{-L}^L \left(1 - \frac{y^2}{L^2}\right) \cos(\lambda_m y) dy = \sum_{n=0}^{\infty} C_n \int_{-L}^L \cos(\lambda_n y) \cos(\lambda_m y) dy$$

$$n \neq m$$

$$= C_m \int_{-L}^L \cos^2(\lambda_m y) dy$$

$$\Rightarrow C_m = \frac{\int_{-L}^L \left(1 - \frac{y^2}{L^2}\right) \cos(\lambda_m y) dy}{\int_{-L}^L [\cos(\lambda_m y)]^2 dy}$$

$$u(y,t) = u_0 \left(1 - \frac{y^2}{L^2}\right) + \phi(y,t)$$

Substitute this into the PDE $\rightarrow \frac{\partial \phi}{\partial t} = \frac{\mu}{\rho} \frac{\partial^2 \phi}{\partial y^2}$

I.C $\phi(y, t=0) = -u_0 \left(1 - \frac{y^2}{L^2}\right)$

B.C $\phi(y=\pm L, t) = 0$

linear, homogeneous
solve by the method of separation of variables

① $\phi(y,t) = Y(y) T(t)$

② $Y T' = \frac{\mu}{\rho} Y'' T \rightarrow \left(\frac{T'(t)}{\frac{\mu}{\rho} T(t)} \right) = \frac{Y''(y)}{Y(y)} = \text{Constant} = -\lambda^2$

$T(t) = A e^{-\frac{\mu}{\rho} \lambda^2 t}$

$Y''(y) + \lambda^2 Y = 0$

$Y(y) = C_1 e^{i\lambda y} + C_2 e^{-i\lambda y} = B_1 \sin \lambda y + B_2 \cos \lambda y$

$$\frac{y}{L}$$

② $Y T' = \frac{\mu}{\rho} Y'' T$

$\sin \lambda y = \frac{e^{i\lambda y} - e^{-i\lambda y}}{2i}$, $\cos \lambda y = \frac{e^{i\lambda y} + e^{-i\lambda y}}{2}$ Symmetry $B_1 = 0$

$\cos \lambda L = 0 \rightarrow \lambda L = \frac{\pi}{2}, \frac{3\pi}{2}, \frac{5\pi}{2}, \dots = \frac{(2k+1)\pi}{2}$

③ $\phi = \sum_{k=0,1,2,\dots} C_k e^{-\frac{\mu}{\rho} \left(\frac{(2k+1)\pi}{2L}\right)^2 t} \cos\left(\frac{(2k+1)\pi}{2L} y\right)$ $k=1, 3, 5, \dots$

④ $-u_0 \left(1 - \frac{y^2}{L^2}\right) = \sum_{k=0,1,2,\dots} C_k \cos\left[\left(k+\frac{1}{2}\right)\frac{\pi}{L} y\right]$ $y \equiv \mu/\rho$

For given m

$C_k \int_{-L}^L \left[\cos\left((m+k+1)\frac{\pi}{L} y\right) + \cos\left((m-k)\frac{\pi}{L} y\right) \right] dy$

$= C_m \frac{1}{2} (2L) = L C_m = -u_0 \int_{-L}^L \left(1 - \frac{y^2}{L^2}\right) \cos\left((m+\frac{1}{2})\frac{\pi}{L} y\right) dy$

$C_m = -u_0 \int_{-L}^L \left(1 - \frac{y^2}{L^2}\right) \cos\left((m+\frac{1}{2})\frac{\pi}{L} y\right) dy$

$= -\frac{u_0}{\pi(m+\frac{1}{2})} \int_{-L}^L (1 - \frac{y^2}{L^2}) d \sin\left((m+\frac{1}{2})\pi \theta\right)$

$= -\frac{u_0}{\pi(m+\frac{1}{2})} \left[- \int_{-1}^1 \sin\left((m+\frac{1}{2})\pi \theta\right) (-2\theta) d\theta \right]$

$= +\frac{u_0}{\pi^2(m+\frac{1}{2})^2} \int_{-1}^1 (-2\theta) d \cos\left((m+\frac{1}{2})\pi \theta\right)$

$= \frac{u_0}{\pi^2(m+\frac{1}{2})^2} \left[- \int_{-1}^1 2 \cos\left((m+\frac{1}{2})\pi \theta\right) d\theta \right]$

$= \frac{u_0}{\pi^2(m+\frac{1}{2})^2} (-2) \left[\sin\left((m+\frac{1}{2})\pi\right) - \sin\left(-\left(m+\frac{1}{2}\right)\pi\right) \right] = -\frac{4 u_0 (-1)^m}{\pi^3 (m+\frac{1}{2})^3}$

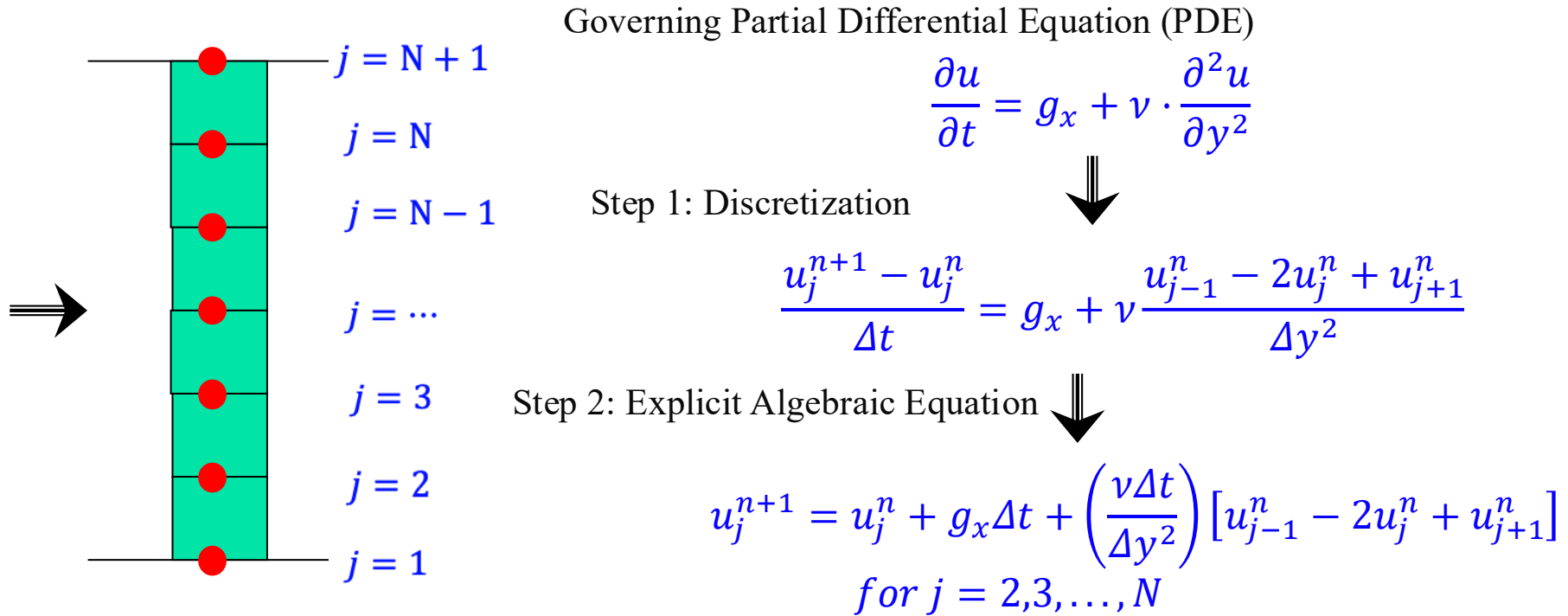
Question:

Why do we need to check numerical simulation results?

If we do not have an analytical solution, how do we check
if our numerical-simulation results are correct?

3. Numerical algorithm: finite-difference method (one of many choices)

Distribution of lattice nodes & Implementation of boundary conditions



A PDE has been converted into an explicit algebraic equation

3. Numerical algorithm: simulation parameters

Finite Difference Scheme

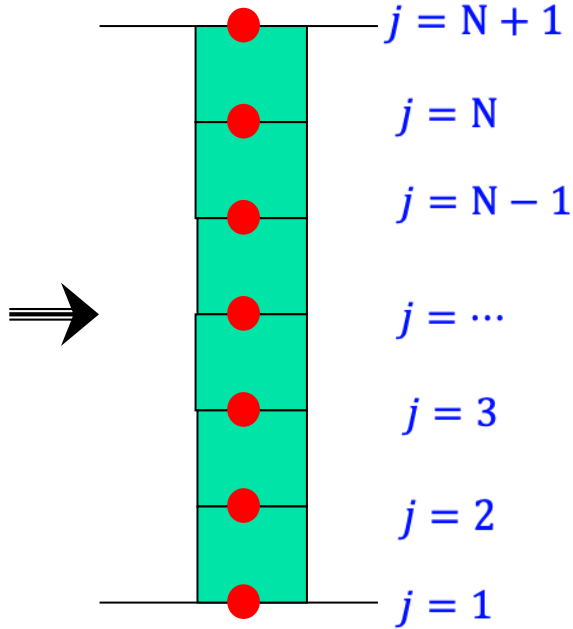
$$u_j^{n+1} = u_j^n + g_x \Delta t + \left(\frac{\nu \Delta t}{\Delta y^2} \right) [u_{j-1}^n - 2u_j^n + u_{j+1}^n]$$

for $j = 2, 3, \dots, N$

Inputs: $N, u_0 = 1.0, L = 1.0, \nu = 0.1$

$$\Delta y = \frac{2L}{N}, \quad \frac{\nu \Delta t}{\Delta y^2} = 0.32 \rightarrow \Delta t, \quad g_x \Delta t = \frac{2\nu \cdot u_0}{L^2} dt = \frac{2.56}{N^2}$$

View results every: $\frac{\nu \cdot ntime \cdot \Delta t}{L^2} = 0.2048$



4. Realization of computation: a computer code

Program main

```
implicit none
! on wall implementation, center difference
! Euler time integration
integer,parameter :: N=80, Ntime=1024
real*8,parameter :: dt = 0.002d0
integer,parameter :: N=40, Ntime=256
real*8,parameter :: dt = 0.008d0
integer,parameter :: N=20, Ntime=64
real*8,parameter :: dt = 0.032d0

real*8,parameter :: u0=1.0d0, aL=1.0d0, anu=0.1
! u0 maximum velocity,aL channel half width,anu kinematic viscosity
! body force is then g = 2*nu*u0/aL^2
real*8, dimension(1:N+1) :: uold, u, uana
real*8 :: theta0, theta1, theory1,ss,ycc,CFL
integer :: j,it,nsteps,k
real*8 :: dy,pi,t,Tend

pi = 4.0d0*atan(1.0d0)
dy = 2.d0*aL/real(N)
! initial condition
u = 0.0d0
uold = u
CFL = dt*anu/dy/dy

write(*,*) 'anu, dt, aL, dy, CFL=', anu, dt, aL, dy, CFL
t = 0.d0
Tend = 3.0d0*aL*aL/anu
nsteps = Tend / dt
write(*,*) 'Tend, dt, Ntime, nsteps=', Tend, dt, Ntime, nsteps
```

```
do it=1, nsteps

do j=2,N
u(j) = uold(j) + CFL*(uold(j-1)-2.d0*uold(j)+uold(j+1)) &
+ dt*2.d0*u0*anu/aL**2.d0
end do

do j=2,N
uold(j) = u(j)
enddo

t = t +dt
! print out solutions
if(mod(it,Ntime).eq.0) then
! analytical solution with the same N
do j = 1,N+1
ycc = real(j-1)*dy - 1.0d0
theory1 = (1.d0- ycc**2.d0)
! I used 100 terms
do k =0,100
ss = (0.5d0+real(k))*pi
theta0 = 4.0d0*(-1.0)**k/(pi*(real(k)+0.5d0))**3.d0
theta1 =theta0 *exp(-ss*ss*anu*t)*cos(ss*ycc)
theory1 = theory1 - theta1
end do

write(55,100)ycc,theory1,u(j)/u0,abs(u(j)-theory1)/theory1
end do
!!!!!!!!!
end if

100 format(2x, 4f16.12)
enddo
end program
```

A DeepSeek demonstration

梁文锋
1985年出生



Ask DS: Explain the code line by line to my students

DS does a great job!

The output file is saved in 1.Ask_DS_to_explain_my_Code_to_students.pdf

I then ask DS to convert my Fortran code to C++ code. It only took a few seconds.

I compile the C++ code by

g++ Channel_FD_Cpp_DScode.cpp [My MacBook has Apple clang version 15.0.0]

The c++ code runs fine, and output the identical results.

I am very impressed!

Als, Google AI (Gemini)



And, it is free.

DeepSeek explanation

Program main

This declares the start of the Fortran program named main.

implicit none

This ensures that all variables must be explicitly declared. It prevents Fortran from implicitly assigning types to variables, which helps avoid errors.

integer,parameter :: N=20, Ntime=64

N=20 : The number of grid points in space.

Ntime=64 : The number of time steps between printing results

real*8,parameter :: dt = 0.032d0

dt=0.032d0 : The time step size for the simulation.

do j=2,N

u(j) = uold(j) + CFL*(uold(j-1)-2.d0*uold(j)+uold(j+1)) &
+ dt*2.d0*u0*anu/aL**2.d0

This updates the velocity field u using a finite difference scheme:

- The term $CFL*(uold(j-1)-2.d0*uold(j)+uold(j+1))$ approximates the second derivative (diffusion term).
- The term $dt*2.d0*u0*anu/aL**2.d0$ represents a body force.

end do

do j = 1,N+1

ycc = real(j-1)*dy - 1.0d0
theory1 = (1.d0- ycc**2.d0)

do k =0,100

ss = (0.5d0+real(k))*pi
theta0 = 4.0d0*(-1.0)**k/(pi*(real(k)+0.5d0))**3.d0
theta1 =theta0 *exp(-ss*ss*anu*t)*cos(ss*ycc)

theory1 = theory1 - theta1

end do

This calculates the analytical solution for the velocity profile:

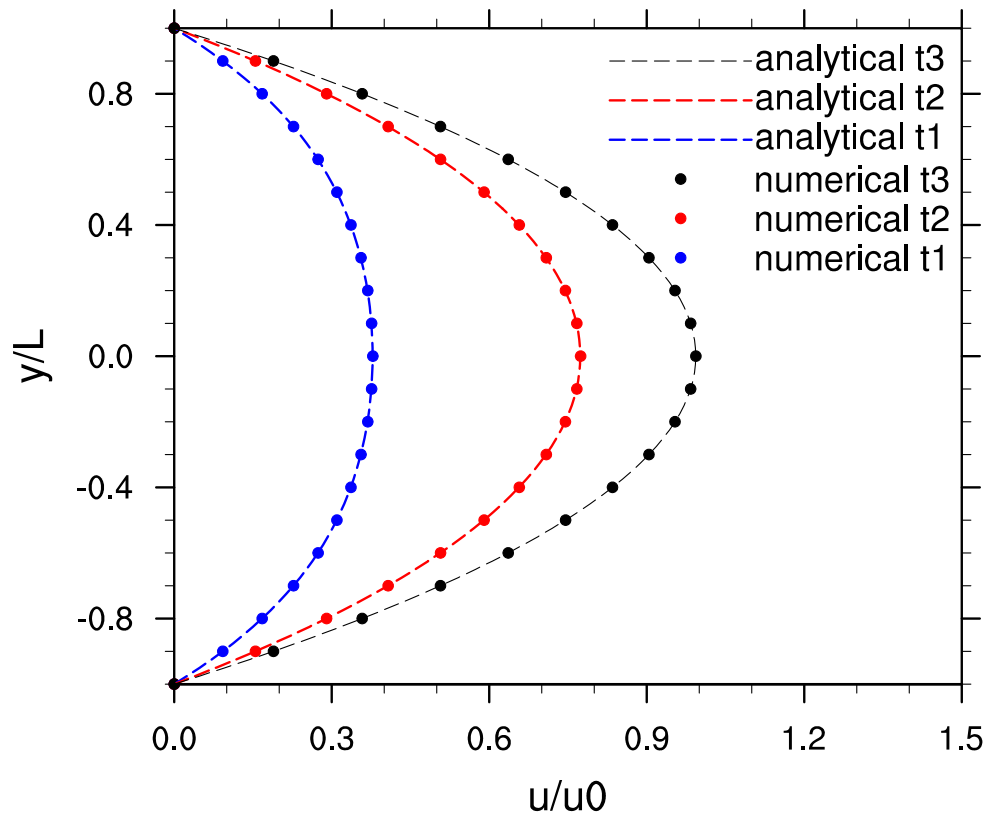
- ycc is the spatial coordinate.
- theory1 is the initial parabolic profile.
- The loop over k adds higher-order terms to approximate the analytical solution.

DeepSeek Summary of my code

This code solves a 1D diffusion equation with a body force using a finite difference method and Euler time integration. It compares the numerical solution with an analytical solution and writes the results to a file at specified intervals. The key steps include:

1. Initializing parameters and arrays.
2. Updating the velocity field using a finite difference scheme.
3. Calculating and comparing the numerical and analytical solutions.
4. Writing results to a file.

5. Verification and validation



$$\frac{v \cdot t_1}{L^2} = 0.2048$$

$$\frac{v \cdot t_2}{L^2} = 0.6144$$

$$\frac{v \cdot t_3}{L^2} = 2.048$$

At this point, I would like to ask you a simple question:

When we ask a computer to calculate something for us, is the computer doing the exact calculation, or is it doing approximate calculation?

Why?

6. Use and interpretation of the computer solution

Make detailed physical observations

How long does it take for the flow to reach steady state?

What is the key physical parameter for this problem?

How does the relative error change with grid resolution?

Will the solution converge to the analytical solution?

What is the truncation error?

Numerical stability?

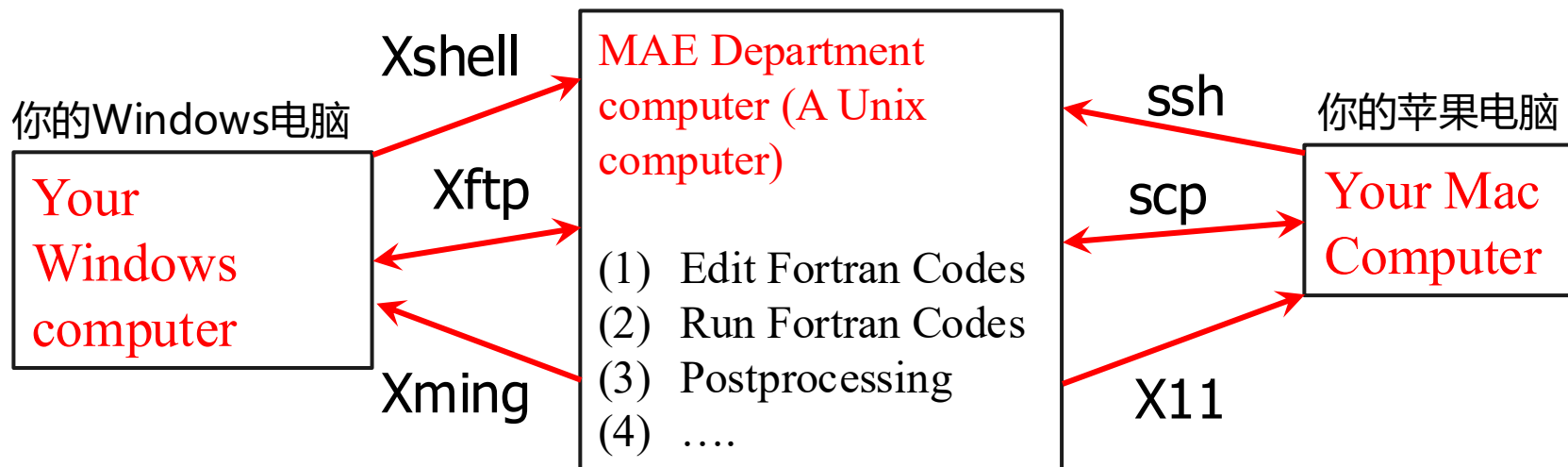
Code optimization?

ZHAO Xiaozheng (Next Monday):

A demo: accessing the department computer

Unix commands, vi editor, using gfortran

力航系的Unix/Linux系统
IP address = 172.18.129.92



`ssh -Y maewanglp@172.18.129.92`

Necessary computer skills

Unix OS basics: <http://www.udel.edu/it/help/unix/using.html>

[Introduction to Unix / Linux](#), a handout you should read

Information on vi editor:

<https://services.udel.edu/TDClient/32/Portal/KB/ArticleDet?ID=531>

Fortran, compiler, library, parallelization

[Introduction to Fortran 90](#), plus two other books are provided.

Learning Fortran by using it (hands-on learning)

Unix computer platform: processor, cache, nodes, memory, etc.

Learning by doing 一边做一边学

Why do we need a computer and programming?

A very simple example

A simple example

If a coin is tossed 1,000 times, what is the probability of having the head exactly 500 times?

Mathematical expression:
$$p = \frac{C(n, k)}{2^{1000}} = \frac{1000 \cdot 999 \cdot 998 \cdot \dots \cdot 503 \cdot 502 \cdot 501}{500 \cdot 499 \cdot 498 \cdot \dots \cdot 3 \cdot 2 \cdot 1 \cdot 4^{500}}$$

Code 1

```
Program main
implicit none
integer, parameter :: n = 1000
integer, parameter :: m = 500
real*4 y
integer i,nh
nh = n/2
y=1.0
do i=1,nh
y=y*(n+1-i)/4.0/(nh + 1 -i)
write(11,*) i, y, alog10(y)
enddo
write (*,*) y
end program
```

1.42297268E+11

Code 2

```
Program main
implicit none
integer, parameter :: n = 1000
integer, parameter :: m = 500
real*8 y
integer i,nh
nh = n/2
y=1.0
do i=1,nh
y=y*(n+1-i)/4.0/(nh + 1 -i)
write(12,*) i, y, dlog10(y)
enddo
write (*,*) y
end program
```

2.5225018178360818E-002

Details matter!

Code 3

```
Program main
implicit none
integer, parameter :: n = 1000
integer, parameter :: m = 500
real*4 y
integer i,nh
nh = n/2
y = 1.0
do i=1,n/4
y=y*(n+1-i)/4.0/(nh+1-i)
y=y*(nh+i)/REAL(i)/4.
write(13,*) i, y, alog10(y)
enddo
write (*,*) y
end program
```

2.52250321E-02

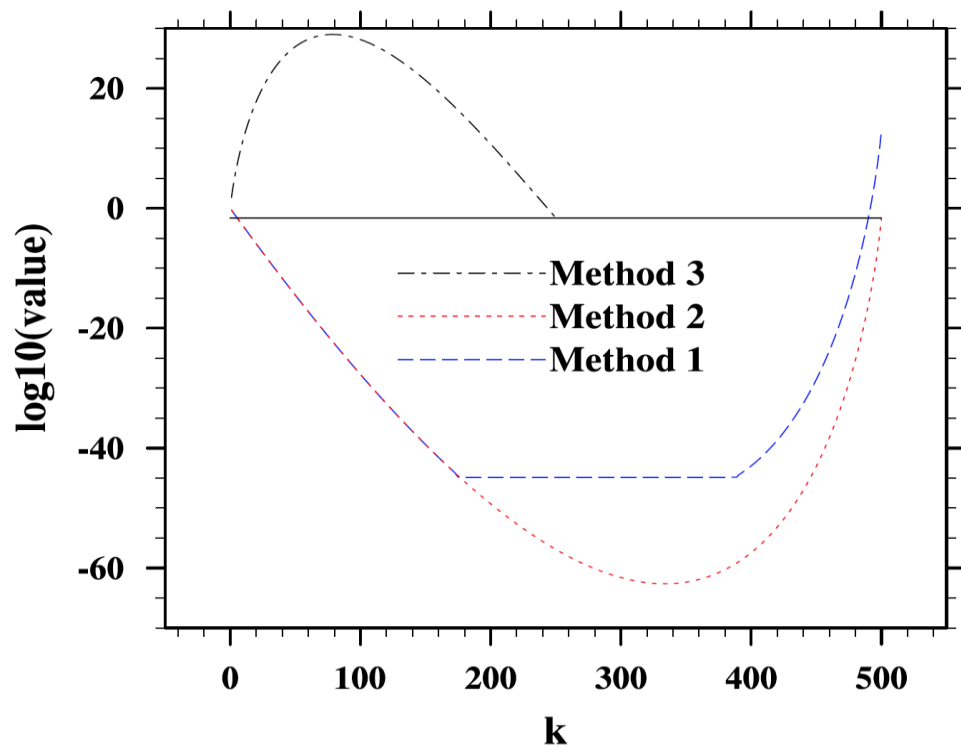
$$C(n, k) \equiv \frac{n!}{k! \cdot (n - k)!}$$

Binomial coefficient

The devil is in the details

Small things in plans and schemes that are often overlooked can cause serious problems later on

Graphically, see the cumulative result



← -44.8534698

Why?

$$\log_{10} y = -44.8534698$$
$$\rightarrow y = 1.40 \times 10^{-45}$$

Smallest subnormal number in single precision

Computer representation of real numbers

bits & bytes

One byte = collection of 8 bits

e.g., 0 1 0 1 1 0 1 0

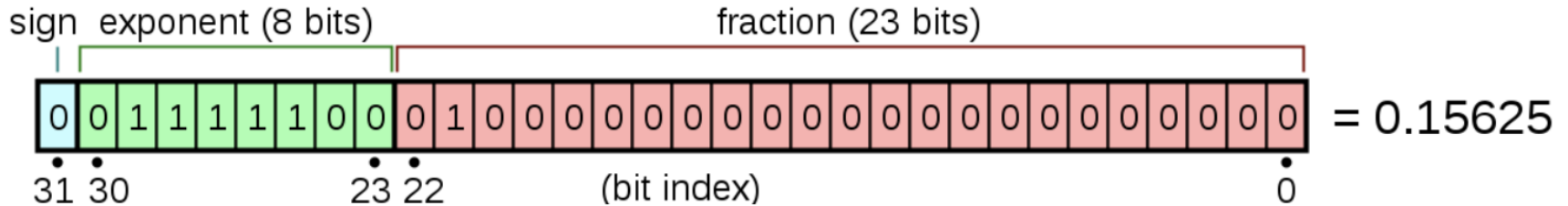
Single-precision = a **binary32** having (4 bytes, 32 bits)

- Sign bit: 1 bit
- Exponent: 8 bits
- Significand precision: 24 bits (23 explicitly stored)

The exponent field is an 8-bit unsigned integer from 0 to 255 in biased form: from -126 to $+127$ (exponents of -127 (all 0s) and $+128$ (all 1s) are reserved for special numbers)

The 24-bit significand precision gives from 6 to 8 significant decimal digits precision [$\log_{10}(2^{24}) \approx 7.225$]

~ 7 significand precision



The code `inputoutput.f90`

! consult

https://en.wikipedia.org/wiki/Single-precision_floating-point_format

!

```
program main range
```

real*4 input

integer::bininput

```
20 read(*,*)input
```

```
write(*,*)'output = ',input
```

```
bininput = transfer(input,bininput)
```

```
write(*,fmt="(b32.32)")binput
```

go to 20

```
end program main range
```

```
-bash-4.2$ ./a.out
```

0.15625

output = 0.15625000

00111100010000000000000000000000

1.4012984643E-45

output = 1.40129846E-45

[illegible]

3.4028234664E38

```
output = 3.40282347E+38
```

0111111101111111111111111111111111

Single-precision examples

Hexadecimal:

0,1,2,3,4,5,6,7,8,9,
a(10),b(11),c(12),d(13),e(14),f(15)

https://en.wikipedia.org/wiki/Single-precision_floating-point_format

0 00000000 000000000000000000000001₂ = 0000 0001₁₆ = $2^{-126} \times 2^{-23} = 2^{-149} \approx 1.4012984643 \times 10^{-45}$
(smallest positive subnormal number)

0 00000000 111111111111111111111111₂ = 007f ffff₁₆ = $2^{-126} \times (1 - 2^{-23}) \approx 1.1754942107 \times 10^{-38}$
(largest subnormal number)

0 00000001 000000000000000000000000₂ = 0080 0000₁₆ = $2^{-126} \approx 1.1754943508 \times 10^{-38}$
(smallest positive normal number)

0 11111110 111111111111111111111111₂ = 7f7f ffff₁₆ = $2^{127} \times (2 - 2^{-23}) \approx 3.4028234664 \times 10^{38}$
(largest normal number)

0 01111110 111111111111111111111111₂ = 3f7f ffff₁₆ = $1 - 2^{-24} \approx 0.999999940395355225$
(largest number less than one)

0 01111111 000000000000000000000000₂ = 3f80 0000₁₆ = 1 (one)

0 01111111 000000000000000000000001₂ = 3f80 0001₁₆ = $1 + 2^{-23} \approx 1.00000011920928955$
(smallest number larger than one)

1 10000000 000000000000000000000000₂ = c000 0000₁₆ = -2
0 00000000 000000000000000000000000₂ = 0000 0000₁₆ = 0
1 00000000 000000000000000000000000₂ = 8000 0000₁₆ = -0

0 11111111 000000000000000000000000₂ = 7f80 0000₁₆ = infinity
1 11111111 000000000000000000000000₂ = ff80 0000₁₆ = -infinity

0 10000000 10010010000111111011011₂ = 4049 0fdb₁₆ $\approx 3.14159274101257324 \approx \pi$ (pi)
0 01111101 01010101010101010101011₂ = 3eaa aaab₁₆ $\approx 0.333333343267440796 \approx 1/3$

x 11111111 100000000000000000000001₂ = ffc0 0001₁₆ = qNaN (on x86 and ARM processors)
x 11111111 000000000000000000000001₂ = ff80 0001₁₆ = sNaN (on x86 and ARM processors)

Examples

Input: 1.0

Binary: 0 01111111 0000000000 0000000000 000

Output: 1.0000000000000000E+00

Binary representation has no round-off error, so the number of significant digits can be very large

Input: 3.14

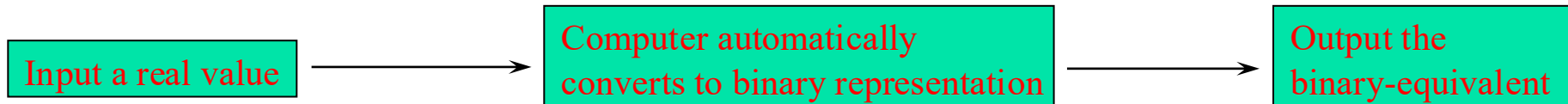
Binary: 0 10000000 1001000111 1010111000 011

Output: 3.140000104904175E+00

Binary representation has a maximum round-off error of $2^{128-127} \cdot 2^{-23} \cdot 0.5 = 0.00000011920928955 \dots$

Actual round off error = 0.000000104904175

Theory / logic works well!



Round-off may occurs

```
program main_range
real*4 input
integer::binput
20 read(*,*)input
write(*,*)'output = '
write(*,100)input
100 format(2x,1PE25.15)
binput = transfer(input,binput)
write(*,fmt="(b32.32)")binput
go to 20
end program main_range
```

$2 \cdot (1 + 2^{-1} + 2^{-4} + 2^{-8} + 2^{-9} + 2^{-10} + 2^{-11} + 2^{-13} + 2^{-15} + 2^{-16} + 2^{-17} + 2^{-22} + 2^{-23}) = 3.140000104904175$

Exact conversion,
Can print out as many
digits as you like.

Computer finds the best binary representation for your real value!

Double precision (8 bytes, 64 bits)

Double-precision =
a **binary64** having

- Sign bit: 1 bit
- Exponent: 11 bits
- Significand precision: 53 bits (52 explicitly stored)

The exponent field is an 11-bit unsigned integer from 0 to 2047 in biased form: from -1022 to +1023

The 53-bit significand precision gives from 15 to 17 significant decimal digits precision ($\log_{10}(2^{53}) \approx 15.95$)

16 significand precision

Quadruple precision (16 bytes, 128 bits)

Quadruple-precision =
a **binary128** having

- Sign bit: 1 bit
- Exponent: 15 bits
- Significand precision: 113 bits (112 explicitly stored)

The exponent field is an 15-bit unsigned integer from 0 to 32767 in biased form: from -16382 to +16383

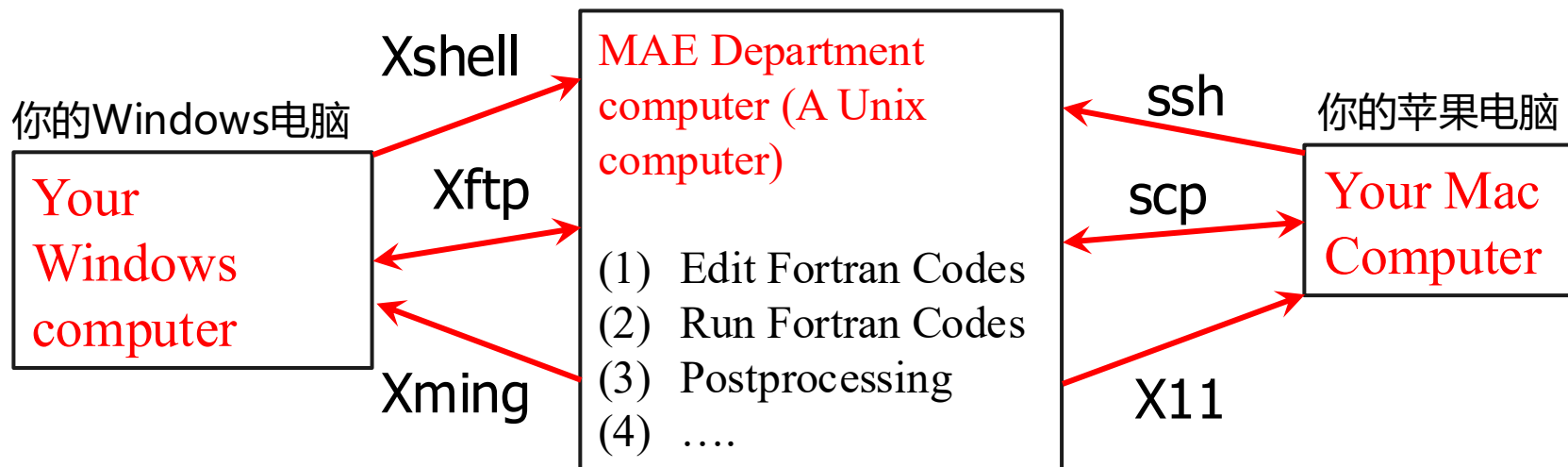
The 113-bit significand precision gives from 33 to 36 significant decimal digits precision ($\log_{10}(2^{113}) \approx \log_{10}(2^{53}) \approx 34.02$)

34 significand precision

Question:

In single-precision representation, what is the magnitude of the computer round-off error when representing the value of π ?

力航系的Unix/Linux系统
IP address = 172.18.129.92



`ssh -Y maewanglp@172.18.129.92`

Introduction to
Computation / Computational fluid dynamics

Computation has always been helpful for routine tasks:

..... when we cannot solve things *analytically*.

Examples:

- Roots of a nonlinear equation

- Numerical differentiation

- Numerical integration

- Interpolation

- Curve fitting / regression

- Optimization

- Solution of an ordinary differential equation

- Solution of a system of partial differential equations (fluid mechanics)

.....

Historical notes on computational methods for fluid flows

First digital computers:

1937 – 1943 (John Vincent Atanasoff and graduate student Cliff Berry)

1943-1946: ENIAC invented by J. Presper Eckert and John Mauchly, U. Pennsylvania

Finite difference / finite volume (since 1928)

Finite element (since 1941)

Finite volume (since 1950s, become more popular in 1980s)

Spectral / pseudo-spectral (Steve Orszag, 1969)

Boundary element

Vorticity based methods

Lattice Boltzmann method (since 1988, evolved from lattice gas automata)

Dissipative particle dynamics (Since 1992, evolved from lattice gas automata)

Smoothed particle hydrodynamics (since 1977)

Last 20 years, many new algorithms for multiphase flows!

Some review papers on the history of CFD

MG Hall (1981) Computational Fluid Dynamics: A revolutionary force in aerodynamics, *AIAA Paper* 81-1014, Palo Alto, CA.

E. Krauss (1985) Computational Fluid Dynamics: Its present status and future direction, *Comput. & Fluids* 13: 239-269.

SG Rubin and JC Tannehill (1992) Parabolized / reduced Navier-Stokes computational techniques. *Annu. Rev. Fluid Mech.* 24: 117-144.

RW MacCormack (1993) A perspective on a quarter century of CFD research. *AIAA Paper* 93-3291-CP, Honolulu, HI.

FT Johnson, EN Tinoco, NJ Yu (2005), Thirty years of development and application of CFD at Boeing Commercial Airplanes, Seattle. *Comput. & Fluids* 34: 1115-1151.

Finite difference method

- Historically, the oldest of the three (finite difference, finite element, finite volume).
- Techniques published as early as 1910 by L. F. Richardson.
- Seminal paper by Courant, Friedrichs and Lewy (1928) derived stability criteria for explicit time stepping. CFL number.
- First ever numerical solution: flow over a circular cylinder by Thom (1933).
- Scientific American article by Harlow and Fromm (1965) clearly and publicly expresses the idea of “computer experiments” for the first time and CFD is born!!
- Advantage: easy to implement.
- Disadvantages: restricted to simple grids and does not conserve momentum, energy, and mass on coarse grids.

First CFD paper

Thom, A., 1933. The flow past circular cylinders at low speeds. *Proceedings of the Royal Society of London. Series A, Containing Papers of a Mathematical and Physical Character*, 141(845), pp.651-669.

The term “computer experiment” was coined

Harlow, F.H. and Fromm, J.E., 1965. **Computer experiments** in fluid dynamics. *Scientific American*, 212(3), pp.104-111.

Finite volume method (FVM)

- First well-documented use was by Evans and Harlow (1957) at Los Alamos and Gentry, Martin and Daley (1966).
- Was attractive because while variables may not be continuously differentiable across shocks and other discontinuities mass, momentum and energy are always conserved.
- FVM enjoys an advantage in memory use and speed for very large problems, higher speed flows, turbulent flows, and source term dominated flows (like combustion).
- Late 70' s, early 80' s saw development of body-fitted grids. By early 90' s, unstructured grid methods had appeared.
- Advantages: basic FV control volume balance does not limit cell shape; mass, momentum, energy conserved even on coarse grids; efficient, iterative solvers well developed.
- Disadvantages: false diffusion when simple numerics are used.

Computer simulation: A third pillar for scientific discovery

What is it?

Why?

Computer simulation: A third pillar for scientific discovery

Theory, experiment, and computer simulation

Why?

- Better computers

- New methods and algorithms

Historical note

Current technology: transistors on integrated circuit

1943-1969, The early days - First computers / supercomputers

1970-1990, The vector computing years

1991- now, The parallel computing years, clusters, scalable, up to 100,000 processors

The future is not known:

Grid computing, distributed computing, or cloud computing, ...

Alternate technologies?

Optical

Biological / Molecular

Quantum

Speed of computer – FLOPS

Floating-point operations per second

10^6 FLOPS → mega FLOPS [reach around 1965]

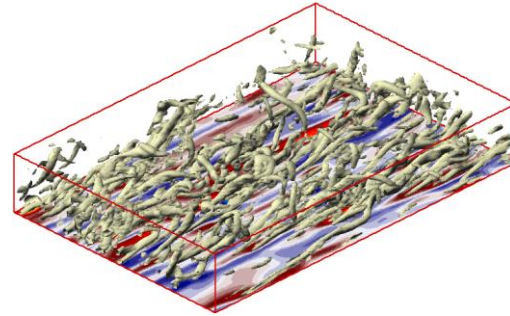
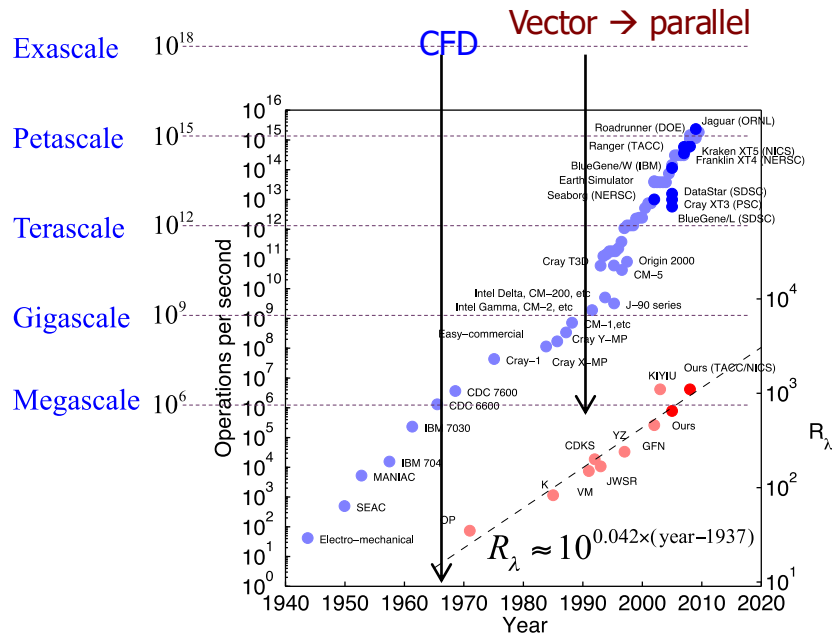
10^9 FLOPS → giga FLOPS [reach around 1990]

10^{12} FLOPS → tera FLOPS [reach around 1997]

10^{15} FLOPS → peta FLOPS [reached around 2008]

10^{18} FLOPS → exa Flops [near future goal]

Evolution of computational power and turbulence simulation



The graph (modified after D.A. Donzis 2010) shows how the increasing computational power over the years enables simulation of turbulent flow at increasing flow Taylor-microscale Reynolds number.

The cost of performing a given calculation was reduced by approximately a factor of 10 every 8 years up through 1995.

After 1995, the world's best computer is 10 times faster approximately every 3.3 years.

→ Speed up in 10 years is a factor of about 1000

→ A problem that requires a year of computing time to solve 10 years ago can now be solved overnight [$365/1000 \sim 0.365$ days ~ 9 hours]

Why computation?

- Traditional science and engineering:
 - ✧ Do **theory** or paper design
 - ✧ Perform **experiments** or build system
- Limitations:
 - Too difficult – build large wind tunnels
 - Too expensive – build a throw-away airplane
 - Too slow – wait for climate or galactic evolution
 - Too dangerous – weapons, drug design, climate experiments
 - Too challenging or impossible to measure – instrumentation not ready
- Solution: Use high performance computer systems to **simulate** the phenomenon and obtain quantitative understanding – **first-principle simulation and numerical experiment**

The cost of computation is decreasing, but the cost of performing experimentation is increasing!

NSF Cyberinfrastructure Framework for 21st Century Science and Engineering (CF21):
Computation is accepted as the third pillar supporting innovation and discovery ...

Computer modeling or computer simulation??

Computer modeling: Reproducing known physics

Examples: Turbulence modeling in engineering, General circulation models,

Computer simulation: generating original data for unknown physics. Also known as computer experiment, direct numerical simulation,

Most computational studies sit in between

Modeling

Typically based on coarse-grained “averaged equations”

Many unresolved physical issues are “parameterized” (as transport coefficients)

- Closures

- Parameterizations

- Models (e.g., subgrid scale models)

Simulations

Typically based on original valid equations

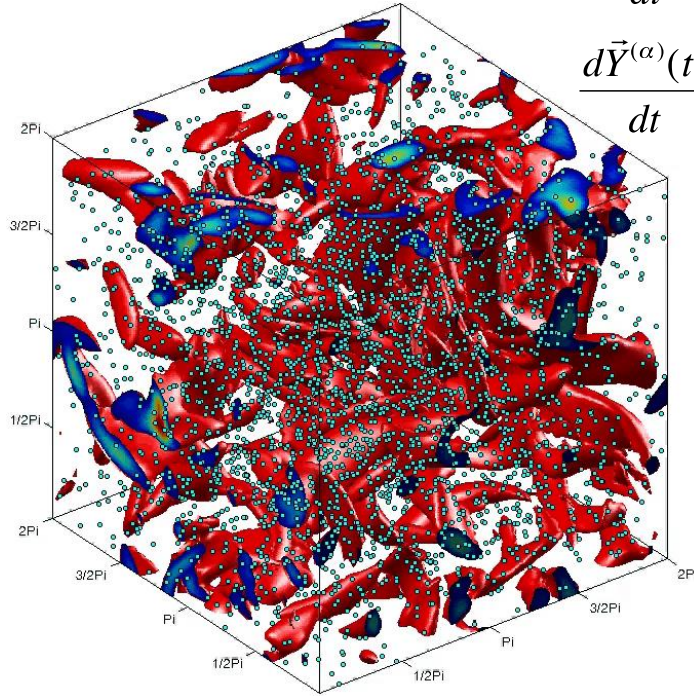
Parameterizations (i.e., subgrid-scale models, particle equation of motion), if there are, have high fidelity

Limited to simple setting or subset of the whole problem

Hybrid Direct Numerical Simulation

$$\frac{d\vec{V}^{(\alpha)}(t)}{dt} = - \frac{\vec{V}^{(\alpha)}(t) - \left[(U(\vec{Y}^{(\alpha)}(t), t) + \vec{u}(\vec{Y}^{(\alpha)}, t)) \right]}{\tau_p^{(\alpha)}} - \vec{g}$$

$$\frac{d\vec{Y}^{(\alpha)}(t)}{dt} = \vec{V}^{(\alpha)}(t)$$



Beyond “point particles”:

Droplet-droplet aerodynamic interactions are considered

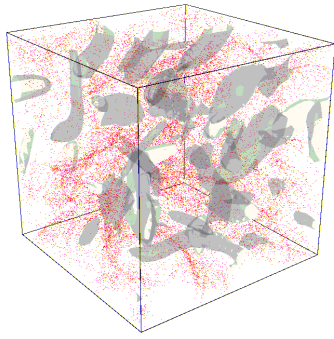
Wang et al., Turbulent collision of inertial particles: Point-particle based, hybrid simulations and beyond. *Int. J. Multiphase flow*, 35 (2009) 854–867.

In this course, we will stress *direct numerical simulations* of viscous flows, turbulence and multiphase flows

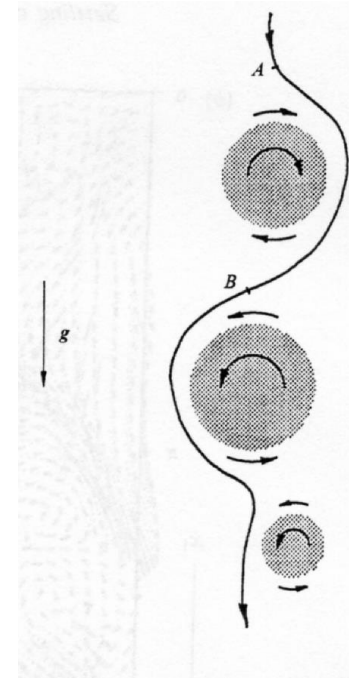
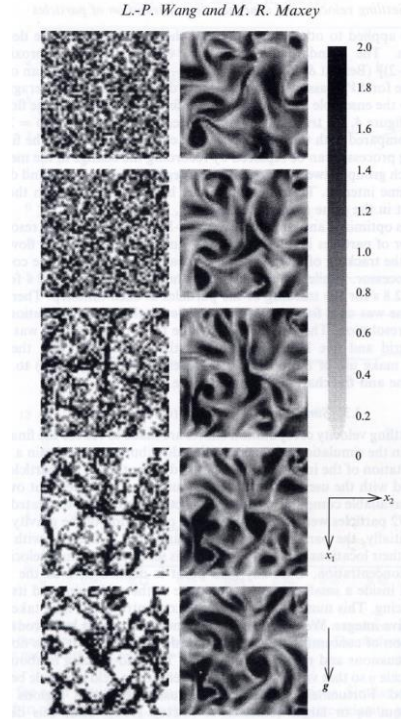
Simulations can lead to discovery

Particle distribution and settling in isotropic turbulence

Wang & Maxey, J. Fluid Mechanics, 256, 27-68, 1993.



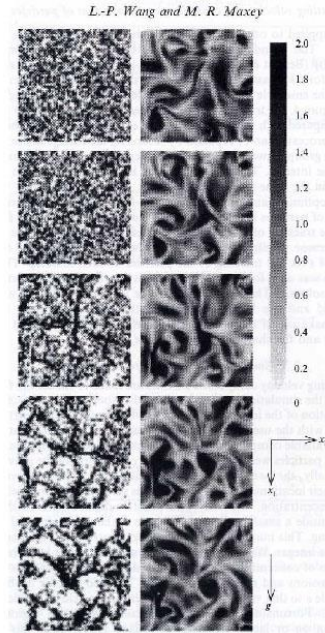
Point-particle based
One-way coupling



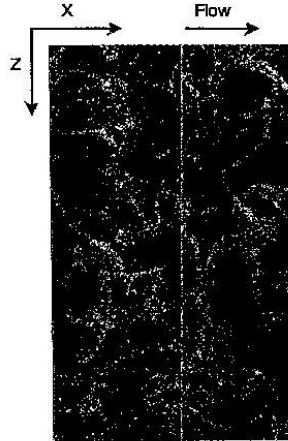
Experimental validations came later ...

**Kolmogorov scaling of
preferential concentration**

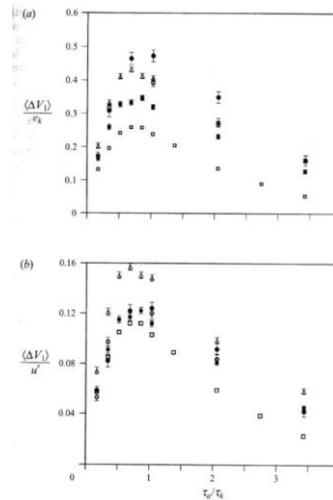
**Enhanced settling
rate**



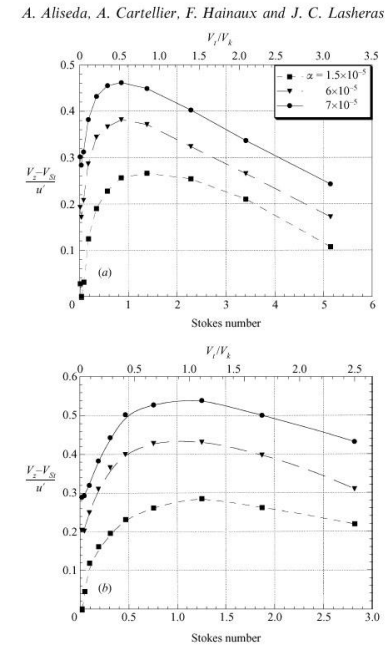
**Fessler, Kulick &
Eaton (1994)**



**Wang &
Maxey
(1993)**



**Wang & Maxey
(1993)**



Aliseda et al. (2002)

Simulations lead to discovery:

The 2013 Nobel prize in chemistry

Martin Karplus, Harvard U., Cambridge, MA, USA

Michael Levitt, Stanford U., Stanford, CA, USA

Arieh Warshel, U. Southern Ca., Los Angeles, CA, USA

The Nobel Prize in Chemistry 2013 was awarded jointly to Martin Karplus, Michael Levitt and Arieh Warshel “for the development of multiscale models for complex chemical systems”

Interactions between atoms → the structure of proteins

The capability of a method is realized only when physical phenomena can be simulated accurately

Verification: The process of evaluating a product to determine whether it meets the initial requirements and design specifications.

Quality Control. Are you building the method right?

Validation: The process of evaluating a product during or at the end of the development process to determine whether it satisfies specified requirements.

Quality Assurance: Are you building the right method?

Applications: Use the method to solve problems at conditions that have not been solved before.

Physics comes first!

The devil is in the details

Small things in plans and schemes that are often overlooked can cause serious problems later on

Comparison of approaches

Approach	Advantages	Disadvantages
Experimental	most realistic	1. equipment required 2. scaling problems 3. tunnel corrections 4. measurement difficulties 5. high operating costs
Theoretical	clean, general	1. restricted to simple geometry & physics 2. usually for linear problems
Computational	OK for nonlinear probs OK for complex physics OK for transient probs	1. truncation errors 2. boundary condition problems 3. computer costs 4. coding errors 5. limited domain size

Summary

Computational fluid dynamics is an interdisciplinary topic

- physical principles (mass, momentum, energy balance)
- mathematical formulation (e.g., PDE)
- numerical methods, algorithms
- analysis and interpretation
- computer science

Computer representation of numbers: bits and bytes

- Binary representation of single-precision / double precision numbers
- Round-off errors

Computer simulation and computer modeling

- Both are needed, for different reasons

Computer simulation is a new tool for discovery

- Verification versus validation
- What does this mean?
- How does this impact the society?

Computer representation of real numbers (bits & bytes)

Fortran programming, Unix OS

$$\cos \alpha \cos \beta = \frac{1}{2} [\cos(\alpha - \beta) + \cos(\alpha + \beta)]$$